

Python for Web Development

Lesson 9: Deployment to Production

Deployment to Production – Study Notes

Key Concepts

1. **Containerization with Docker** – packaging an app and its dependencies into a portable image.
2. **Platform as a Service (PaaS) vs. Infrastructure as a Service (IaaS)** – differences between Heroku (PaaS) and AWS (IaaS/managed services) for production deployments.
3. **Continuous Integration / Continuous Deployment (CI/CD)** – automating build, test, and release pipelines.
4. **Environment configuration & secrets management** – handling variables, credentials, and config per environment (dev / staging / prod).
5. **Observability basics** – logging, metrics, and health checks for a production ready service.

Summary

Deploying an application to production involves turning source code into a reliable, repeatable artifact and then delivering that artifact to a runtime environment that can scale, stay secure, and recover from failure. Docker makes the artifact reproducible by bundling the code, runtime, libraries, and OS level dependencies into a single image that runs the same everywhere. Once containerized, you can choose a hosting model: **Heroku** offers a simple “git push” workflow with managed dynos, while **AWS** provides more granular control through services such as Elastic Container Service (ECS), Elastic Kubernetes Service (EKS), or Elastic Beanstalk.

To keep deployments fast and error free, CI/CD pipelines automate the steps from code commit to production release. A typical pipeline builds the Docker image, runs unit/integration tests, pushes the image to a registry (Docker Hub, ECR, etc.), and then triggers the target platform to pull the new image and restart the service. Proper handling of environment variables, secret stores (Heroku Config Vars, AWS Secrets Manager, Parameter Store), and health check endpoints ensures that the same image can be safely promoted across dev, staging, and prod without code changes. Adding basic observability—structured logs, metrics, and alerts—completes the production readiness loop.

Important Points

- **Dockerfile best practices**
 - Use a lightweight base image (e.g., `python:3.11 slim`, `node:20 alpine`).
 - Leverage multi stage builds to keep final images small.
 - Pin exact versions of OS packages and language dependencies.
 - Add a non root user and set `WORKDIR`.
- **Heroku deployment flow**
 1. `heroku create <app-name>` (or via dashboard).

2. Add a `Dockerfile` or `heroku.yml`.
3. `heroku container:push web --app <app-name>` builds image locally and pushes to Heroku's registry.
4. `heroku container:release web --app <app-name>` releases the image.
 - **AWS deployment options**
 - **ECS/Fargate** – serverless containers; define a task definition, service, and load balancer.
 - **EKS** – managed Kubernetes; use `kubectl`/Helm to apply Deployment manifests.
 - **Elastic Beanstalk (Docker)** – abstracts ECS/EKS; upload a Docker image or `Dockerrun.aws.json`.
 - **CI/CD pipeline skeleton (GitHub Actions example)**

```
```yaml
```

```
name: CI/CD
```

```
on:
```

```
 push:
```

```
 branches: [main]
```

```
jobs:
```

```
 build-and-deploy:
```

```
 runs-on: ubuntu-latest
```

```
 steps:
```

```
 - uses: actions/checkout@v4
```

```
 - name: Set up Docker Buildx
```

```
 uses: docker/setup-buildx-action@v3
```

```
 - name: Log in to registry
```

```
 uses: docker/login-action@v3
```

```
 with:
```

```
 registry: ghcr.io
```

```
 username: ${{ github.actor }}
```

```
 password: ${{ secrets.GITHUB_TOKEN }}
```

```
 - name: Build & push image
```

```
 uses: docker/build-push-action@v5
```

```
 with:
```

```
 context: .
```

```
 push: true
```

```
 tags: ghcr.io/${{ github.repository }}:latest
```

```
 - name: Deploy to Heroku
```

```
 env:
```

```
 HEROKU_API_KEY: ${{ secrets.HEROKU_API_KEY }}
```

```
 run: |
```

```
 heroku container:login
```

```
 heroku container:push web --app ${{ secrets.HEROKU_APP_NAME }}
```

```
 heroku container:release web --app ${{ secrets.HEROKU_APP_NAME }}
```

```
```
```

- **Secrets & config**

- Never hard code credentials.
- Use platform specific secret stores (Heroku Config Vars, AWS Secrets Manager, Parameter Store, GitHub Actions secrets).
- Load them at runtime (e.g., `process.env.DB\_URL`).
 - **Health checks & graceful shutdown**
- Expose `/health` endpoint that returns `200` when the app can serve traffic.
- In Docker, use `HEALTHCHECK` instruction.
- Handle `SIGTERM` to finish in flight requests before exiting.
 - **Observability**
- **Logging** – send JSON logs to stdout; platforms aggregate them (Heroku Logplex, CloudWatch).
- **Metrics** – expose Prometheus metrics (`/metrics`) or use AWS CloudWatch Container Insights.
- **Alerting** – set thresholds on error rates, latency, CPU/memory usage.

Examples

1b ã Dockerizing a simple Express API and deploying to Heroku

Dockerfile

```dockerfile

#### ---- Build stage ----

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build # if using TypeScript or bundler
```

#### ---- Runtime stage ----

```
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist # compiled code
EXPOSE 8080
ENV NODE_ENV=production
USER node
CMD ["node", "dist/index.js"]
```
```

Deploy

```bash

heroku login

heroku create my-express-api

```
heroku container:push web --app my-express-api
heroku container:release web --app my-express-api
heroku open --app my-express-api
...
```

\*Explanation\*: The multi stage build keeps the final image ~30 /MB. `heroku container:push` builds the image locally, pushes it to Heroku's private registry, and `release` makes the new version live. Environment variables like `DATABASE\_URL` are added via `heroku config:set DATABASE\_URL=...`.

---

## 2p ã CI/CD pipeline that builds a Docker image, pushes to AWS ECR, and updates an ECS service

**\*\*GitHub Actions workflow (`.github/workflows/deploy.yml`)\*\***

```yaml

name: Deploy to AWS ECS

on:

push:

branches: [main]

jobs:

deploy:

runs-on: ubuntu-latest

permissions:

id-token: write # for OIDC auth to AWS

steps:

- uses: actions/checkout@v4

- name: Configure AWS credentials (OIDC)

uses: aws-actions/configure-aws-credentials@v4

with:

role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsECR

aws-region: us-east-1

- name: Login to Amazon ECR

id: ecr-login

uses: aws-actions/amazon-ecr-login@v2

- name: Build, tag, and push image

env:

ECR_REGISTRY: \${ steps.ecr-login.outputs.registry }

IMAGE_TAG: \${ github.sha }

run: |

docker build -t \$ECR_REGISTRY/my-app:\$IMAGE_TAG .

docker push \$ECR_REGISTRY/my-app:\$IMAGE_TAG

- name: Deploy to ECS

uses: aws-actions/amazon-ecs-deploy-task-definition@v2

with:

task-definition: ecs-task-def.json

```
service: my-ecs-service
cluster: my-ecs-cluster
image: ${{ steps.ecr-login.outputs.registry }}/my-app:${{ github.sha }}
```

...

***Explanation*:** The workflow authenticates to AWS via OIDC (no static keys), builds the Docker image, pushes it to ECR, then updates the ECS service to use the new image tag. The task definition (`ecs-task-def.json`) can contain a placeholder like `"image": "REPLACE_IMAGE"` that the action swaps out automatically.

Quick Review

1. ****What are two main advantages of using a multi stage Docker build for production images?****
2. ****Contrast Heroku's deployment model with AWS ECS/Fargate in terms of operational responsibility.****
3. ****Name three places you can store secrets for a production deployment and explain a best practice for accessing them at runtime.****
4. ****In a CI/CD pipeline, why is it important to run tests *before* pushing a Docker image to a registry?****
5. ****What does a Docker `HEALTHCHECK` directive do, and how does it help orchestration platforms?****

Further Reading

- Docker Official Documentation – **Best practices for writing Dockerfiles**
- Heroku Dev Center – **Container Registry & Runtime**
- AWS Whitepapers – **Running Containers on Amazon ECS** and **Amazon EKS Best Practices**
- “Continuous Delivery” by Jez Humble & David Farley (chapters on pipelines & deployment strategies)
- Observability: **The Log, Metric, Trace Triangle** (Grafana Labs blog)
